

Phase 1 authentication deployment

Verification boundary

This implementation was checked with mocked database/API tests and local TypeScript/lint checks. No production database was inspected, reset, baselined or migrated. Schema-to-schema migration generation does not connect to a database. Prisma's CLI may load its usual environment configuration; secret files were not read or printed by the implementation work.

PostgreSQL migration rollout

The repository previously used db push and had no migration history. `prisma/baseline.postgres.prisma` preserves the pre-auth schema. `20260920000000_baseline` creates that schema; `20260920001000_auth` only adds the active-user flag, sessions, refresh-token history, indexes and foreign keys. MongoDB is unchanged.

1. Back up the target PostgreSQL database and review both SQL files with the deployment owner. Inspect existing migration metadata and compare actual schema to the baseline using approved read-only access. This has NOT been performed here.
2. For an existing database, resolve discrepancies first. Only when it matches the baseline, run from backend: `npm run prisma migrate resolve --applied 20260920000000_baseline --schema=prisma/schema.postgres.prisma`. This writes migration metadata; do not run it blindly or apply baseline CREATE statements over existing tables.
3. Run `npm run prisma migrate deploy --schema=prisma/schema.postgres.prisma` to apply the additive auth migration. For a genuinely empty database, skip resolve and deploy both migrations.
4. Run `npm run build --workspace=backend` from the root in a clean deployment environment, then start with the required configuration. Never use reset or db push to deploy this phase. Stop processes holding generated Prisma engine DLLs before generating on Windows.
5. Verify real PostgreSQL concurrent refresh/replay, rollback on replacement failure, logout and ownership in staging before release. Mock transaction tests are not a substitute for PostgreSQL locking tests.

Existing user provenance gate

No existing password formats or account provenance have been verified. Login accepts only Argon2id hashes; it does not accept plaintext or silently reinterpret legacy passwords. Audit accounts through an approved process and arrange a verified reset/import flow for unsupported credentials (reset/email verification are not implemented). Review legacy accounts and roles before exposure: the additive active flag defaults to true for compatibility. Audit email normalization collisions and normalize existing emails before enabling registration; new registrations trim/lowercase emails, but the existing database unique constraint is case-sensitive. Do not merge ambiguous accounts automatically.

Required environment

- `AUTH_JWT_SECRET`: independently generated, high-entropy base64url secret of at least 32 random bytes; no default. Store in the deployment secret manager.
- `AUTH_JWT_ISSUER` and `AUTH_JWT_AUDIENCE`: explicit stable nonempty identifiers.

- AUTH_ALLOWED_ORIGINS: comma-separated exact HTTPS browser origins without trailing slash; empty disables browser auth. Wildcards and HTTP origins are rejected.
- Existing PostgreSQL and MongoDB connection settings remain unchanged. Do not log credentials, bearer tokens, cookies, refresh tokens or auth request bodies at the reverse proxy/APM layer.

The server fails startup on invalid auth configuration. JWTs use HS256, issuer/audience checks and a 15-minute lifetime. Every protected request checks the backing session and active user. Sessions have an absolute 30-day lifetime. Refresh tokens are 32 random bytes, stored only as SHA-256 hashes. Rotation consumes and replaces in a PostgreSQL transaction under a session-row lock. Replay commits session revocation. Clients must serialize refresh calls; retries with a consumed token intentionally revoke the session. Logout revokes immediately for subsequent requests, not requests already in flight.

Client contract

- POST /api/auth/register: email, password, optional name. Role and unknown fields are rejected. Password is 15–128 Unicode code points and never trimmed. Frontend first/last-name composition is deferred.
- POST /api/auth/login: email and password; generic invalid-credentials errors.
- POST /api/auth/refresh: native JSON refreshToken or browser cookie, never both.
- POST /api/auth/logout and /logout-all: bearer access token and empty JSON body.
- GET /api/users/me and PATCH /api/users/me: bearer token; PATCH permits name only (string or null).
- Native clients send X-Auth-Client: native, no browser Origin/cookies, and receive accessToken, refreshToken, expiresIn. Store refresh credentials using native secure storage in the deferred frontend phase.
- Browsers must send an allowed Origin, X-CSRF-Protection: 1 and credentials-enabled requests on all auth POSTs. Refresh is only in a Secure, HttpOnly, SameSite=Strict, Path=/ __Host- cookie; JSON never returns a browser refresh token. Keep access tokens in memory. Browser signals cannot opt into native mode. SameSite=Strict requires a same-site HTTPS deployment (prefer frontend/API on the same site); unrelated-site deployments are intentionally unsupported.
- Error responses retain string **error** and auth/validation errors add **code**. Sensitive responses are no-store.

Operational limitations

Keep trust proxy false unless a separate reviewed trusted-proxy configuration is introduced. Current IP quotas use in-process memory: restart resets them, multiple replicas require a shared limiter/gateway, and reverse-proxied clients otherwise share the proxy IP. Auth throttles run before the 16 KB parser; scan keeps its independent quota and 10 MB image allowance. AI analysis is authenticated and globally rate limited, but deployment cost controls remain necessary.

Retain consumed refresh history for the full session lifetime to detect replay. A scheduled, reviewed cleanup can delete expired sessions (cascading token history) after an operational retention margin. Do not prune live-session consumed tokens. No cleanup job is installed by this phase.

POST /api/telemetry intentionally retains the firmware contract and has no user JWT requirement. Device existence is NOT device authentication: anyone knowing an ID can forge telemetry. Keep ingestion behind suitable network restrictions until a separately approved device-authentication rollout. Hardware files were not changed.

Frontend integration, secure token storage, navigation/session state, password recovery and the reported Expo manifest-version mismatch remain deferred. This phase is not a claim of production readiness or deployment completion.